

Prim's Algorithm

The Prim's Algorithm is a greedy pathfinding and graph algorithm. It is used to find the MST or the Minimum Spanning Tree of a weighted, connected, and an undirected graph. The goal of the Prim's algorithm is to connect the vertices in the graph while minimizing the edge weight and to avoid the cycles. The algorithm starts by a vertex, then it repeats with the edge that has the smallest possible weight. Which then connects to a visited vertex to an unvisited vertex until all of the nodes in the graph make up a spanning tree.

When you use this approach it will give you an efficient and optimal solution in order to minimize the total of connecting nodes. I would say an advantage of Prim's is the efficiency when you have to work with dense graphs using queues. It also allows you to have an optimal minimum spanning tree solution. A downfall of the algorithm does come with its limitations. One is that it cannot properly handle disconnected graphs very well because they have to be modified. It also doesn't look for the shortest path or for graphs that contain the negative cycles that Dijkstra's or Bellman-Ford do.

Pseudocode

Start

Initialize parent, twokey, and mst arrays

Set all twokey values to infinity

Set all mst values to false

Set twokey[0] = 0

Set parent[0] = -1

Repeat

Find vertex with minimum twokey value

add vertex to the MST

For each adjacent vertex

If edge exists AND vertex not in MST

AND the edge weight is smaller than twokey

Update parent

Update twokey

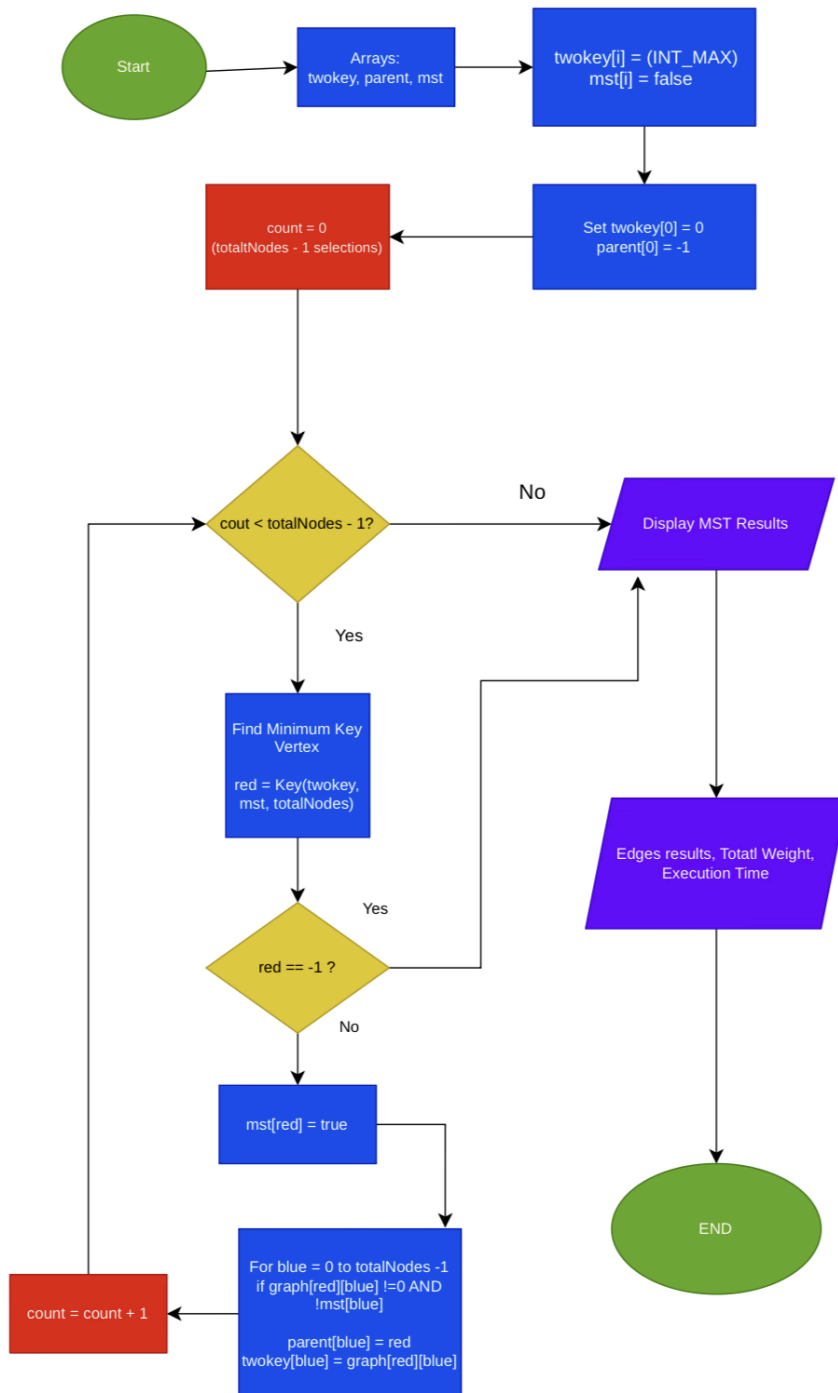
Display MST edges

Display total weight

Display execution time

End

Flow Chart



Test

Case 1: 5x5

For my first test, I did a 5x5 matrix. The 5x5 graph is what allowed me to understand the logic of the algorithm. It also let me see if it was working. It had selected the edges, then showed the total weight ended up being 16. The results showed that the algorithm was working and selecting the lowest-weight edges while also avoiding the cycles. The edge between nodes 3 and 4 had a weight of 9. So it was not chosen because node 3 already had a smaller edge of weight 6. The programs execution run time was 2,084 nanoseconds, and this showed that the graph was running correctly.

```
{ 0, 2, 0, 6, 0 },  
{ 2, 0, 3, 8, 5 },  
{ 0, 3, 0, 0, 7 },  
{ 6, 8, 0, 0, 9 },  
{ 0, 5, 7, 9, 0 }
```

```
=====
```

```
Welcome to the Prim Algorithm  
Prim's Algorithm: MST Results
```

```
=====
```

Edge	Weight
------	--------

0 - 1	2
-------	---

1 - 2	3
-------	---

0 - 3	6
-------	---

1 - 4	5
-------	---

[INFO] Total MST Weight is: 16

[INFO] Total Execution time is: 2084 nanoseconds

```
=====
```

Program ended with exit code: 0

Case 2: 10x10

In my second case I used a larger matrix that was 10x10. In this matrix I wanted to test how my program could handle having more vertices and different edge connections. When it was running most of the results had shown it found the low weighted edges. When it shows node 9 it had a weight of 0, which showed me it didn't handle it well because it was unreachable. It had a weight of 0 because node 9 wasn't connected to any other vertex. The execution time was still 5,584 nanoseconds and this showed that my algorithm was still running correctly. In order to

connect node 9 with node 8, I needed to add a weight. I added a weight of 3 at the end of row 8 and another in row 9 at column 8 so it would stay undirected. This allowed node 9 to be connected with node 8.

```
{0, 4, 0, 0, 0, 0, 0, 8, 0, 0},
{4, 0, 8, 0, 0, 0, 0, 11, 0, 0},
{0, 8, 0, 7, 0, 4, 0, 0, 2, 0},
{0, 0, 7, 0, 9, 14, 0, 0, 0, 0},
{0, 0, 0, 9, 0, 10, 0, 0, 0, 0},
{0, 0, 4, 14, 10, 0, 2, 0, 0, 0},
{0, 0, 0, 0, 0, 2, 0, 1, 6, 0},
{8, 11, 0, 0, 0, 0, 1, 0, 7, 0},
{0, 0, 2, 0, 0, 0, 6, 7, 0, 0}, // {0, 0, 2, 0, 0, 0, 6, 7, 0, 3},
{0, 0, 0, 0, 0, 0, 0, 0, 0, 0} // {0, 0, 0, 0, 0, 0, 0, 0, 3, 0}
```

Welcome to the Prim Algorithm

Prim's Algorithm: MST Results

Edge	Weight
------	--------

0 - 1	4
-------	---

1 - 2	8
-------	---

2 - 3	7
-------	---

3 - 4	9
-------	---

2 - 5	4
-------	---

5 - 6	2
-------	---

6 - 7	1
-------	---

2 - 8	2
-------	---

0 - 9	0
-------	---

[INFO] Total MST Weight is: 37

[INFO] Total Execution time is: 5584 nanoseconds

Program ended with exit code: 0

Results Table

Case	Nodes	Total MST Weight	Execution Time	Status
Test Case 1	5	16	2,084 ns	Pass
Test Case 2	10	37	5,584 ns	Pass

Time complexity

My program uses an adjacency matrix and checks all of the vertices to find the minimum key value. The time complexity ends up being $O(V^2)$. The vector parent, twokey, and mst are what is helping to store all the MST connections, minimum edge weights, and the visited vertices. Overall I believe my algorithm does work but I should improve it to better handle the disconnected nodes.